

QuickCast: Forecasting Business Intelligence Made Easy

Wyatt Keller

Part 1: Problem and Motivation

Oftentimes a small business has so many tasks to deal with that some just fall through the cracks. These businesses don't have the funds to hire an experienced employee for every task. So, what if the very basic forecasting tasks were done in a very easy UI? A business owner being able to calculate how long they have until they run out of a product, see your most popular products, and see general exploratory statistics quickly is essential for a small business owner to make data-driven decisions.

- What problem are you trying to solve?

How can someone without inventory and forecasting experience calculate relevant and useful business statistics easily?

- Why is this problem interesting or important?

Data is beginning to be more integrated with each and every business and being able to work with that data gives you an edge over your competition. Furthermore, in the age of AI, low employee businesses are becoming much more common. These founders may not be able to cover these bases or may need to hire someone who may be underexperienced for these roles. So, lowering the bar for entry allows these new AI businesses to flourish.

- What type of solution will your application provide?

QuickCast will allow for very easy querying through buttons and choosing which function to use. While it may not be the most in-depth forecast, it will allow you to make data-driven decisions that capture most of the value of a forecast.

- Who will use your application?

Small business owners and entry-level employees will find a good use for this application, as it allows low experience with forecasting or types of calculations to do.

- How will the application benefit them?

Users will be able to calculate useful statistics when they may not be able to on their own or may not know what to go out and calculate.

- What information or services will it provide?

Some examples (non-exhaustive) of statistics it'll provide are:

Total amount of a product sold

Total product sold

Most popular products

Products ranked by quartile

Top customers

How long will a product stay in stock at certain rate

Part 2: Data and Database Design

The data chosen to model the database is found here:

<https://www.kaggle.com/datasets/shandeep777/retail-supply-chain-sales-dataset>

It is an excel-based flat file dataset with records of a business about all of their orders, where they went to, and more.

Data Description

The data is record based and is highly denormalized. The main data table, “Retails Order Full Dataset” consists of 23 attributes, while the secondary table “Calendar Table” consists of 12 attributes.

The attributes and descriptions of “Retails Order Full Dataset” are listed below:

- Row ID: The unique identifier for each record
- Order ID: The unique identifier for each order
- Order Date: The date of the order
- Ship Date: The date the order was shipped
- Ship Mode: What class or priority the shipping was for the order
- Customer ID: The unique identifier for each customer
- Customer Name: The name of the customer
- Segment: The type of entity that the customer is (consumer, home office, corporate)

- Country: The country the order is to
- City: The city the order is to
- State: The state the order is to
- Postal Code: The postal code of where the order is to
- Region: The region of where the order is to
- Retail Sales People: Personal of who was involved in the transaction
- Product ID: The unique identifier for each product
- Category: The general category that a product belongs to
- Sub-Category: The specific sub-category that a product belongs to
- Product Name: The name of the product
- Returned: Whether or not the product was returned
- Sales: The amount of money within an order for a specific product
- Quantity: The amount of a specific product that was sold in an order
- Discount: How much a product within an order was discounted
- Profit: The money profited from a specific product within an order

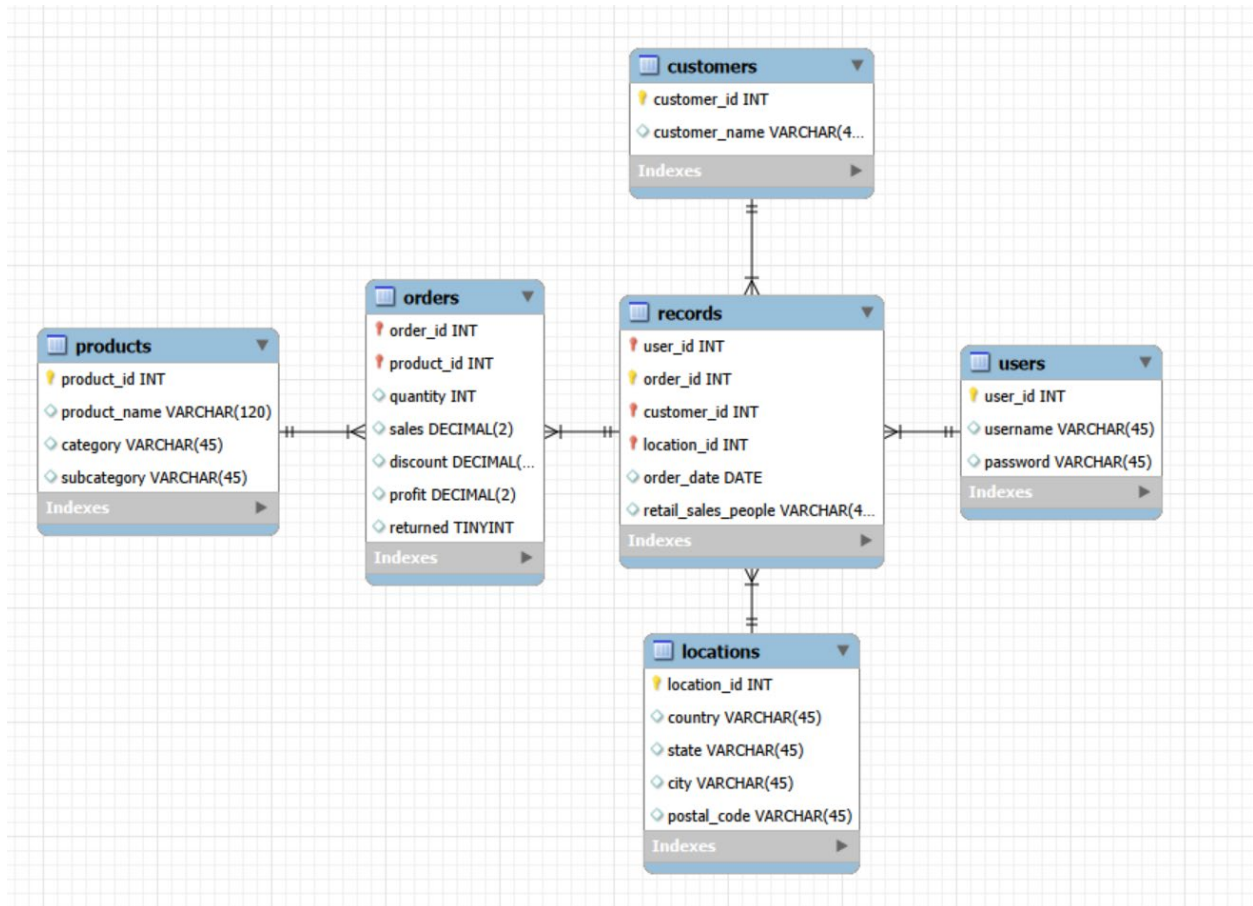
The attributes and descriptions of “Calendar Table” are listed below:

- Date: A specific date on the table
- Year: The year of a date
- Quarter: The numerical quarter of a date
- Quarter (Q): The full quarter of a date
- Quarter & Year: The year and full quarter of a date
- Month: The month of a date
- Month Name: The abbreviated name of a month of a date
- Month & Year: The abbreviated name of a month of a date and the year of the date
- Week of Year: The week number of a date on a specific date
- Week of Year (W) The full week number of a date in a specific date
- Day of Week: The numerical day of a week in a specific date
- Day Name: The full name of a day of a week in a specific date

There will be a good amount of data cleaning required. Mainly it will be removing irrelevant attributes, removing duplicates, taking away extreme values which don't seem right, making sure all the datatypes are the same, and making sure outliers won't affect the demo analysis too much.

Preliminary Database Design

ER Diagram:



Main entities and Relationships:

The main entities are 6 tables, which are records, customers, users, locations, orders, and products.

The records table includes most of the primary and foreign keys, and is to link to customers, users, locations, and orders. The records table has a many to one relationship with the users table, as 1 user may have many records. The records table has a many to one relationship with the customers table, as a customer can be a part of several orders. The records table has a many to one relationship with the locations tables, as one location can be a part of many records. The records table also has a one-to-many relationship with the orders table as a record can have multiple orders associated with it. Finally, the products table has a one-to-many relationship with the orders table, since each product can be a part of several orders.

Primary Keys and Foreign Keys:

The records table has the primary keys of `user_id` and `order_id`, a combination of these will ensure a unique record within the table. The foreign keys `user_id`, `location_id`, and `customer_id` link to users, locations, and customers table respectively.

The customers table has a singular primary key being `customer_id`, which is used as a foreign key to the records table.

The users table has a singular primary key being `user_id`, which is used as a foreign key for the records table.

The locations table has a singular primary key being `location_id`, which is used as a foreign key for the records table.

The orders table has the primary keys of `order_id` and `product_id`, which are also foreign keys to the records table and products table respectively. As in, these keys borrowed from the products and records table allow for a unique key for each entry in the orders table.

The products table has a singular primary key being `product_id`, which is used as a foreign key for the orders table

Design Choices Explanation:

The original design goal was for a star schema, but as I start working through the diagram, it would cause needless duplication. I intended for the records table to be central, at first intended to be the facts table, but then due to there being multiple products within each order, this would cause the records table data to be duplicated for each product. This is why it links to the orders table which has the relevant metrics for each product within the orders table, so that no data is duplicated. Interestingly, from the data it appears that for every order there is only one order date and only one person who handled that order, which enables me to put it on the records table.

Having a user's table enables me to simplify the backend a lot. Since I want to have Flask to be able to help me make a login system, I would require a way to store different users data. The users table works as a way to store the information for the user, while also being able to work as an identifier within the records table. Instead of making a database instance for each user, I want to label the user data for the individual user, so that the backend complexity is reduced but a lot. I'll be able to just filter using the `user_id`.

An important thing to note is that the products and locations tables can be normalized with the creation of 2-3 more tables. I have currently decided against this as it would needly increase the complexity of the storing of the data and queries of the data, and as this is meant to be a small-scale BI tool, the extreme levels of normalization are not needed. Though if it is required for it to

be normalized to level 3 or 4, then please let me know through feedback, as I believe that the current database schema is only normalization level 2.

This is likely not the end database schema, as I would need to adjust it as I actually import the data in, and ensure that the normalization works as intended. Furthermore, I need to check for typing within the dataset and may need to do conversions.

Part 3: Team and Project Organization

Team Roles:

Considering this team has 1 member, Wyatt Keller, his role will be in every part of the project. I didn't particularly see anyone with the same interest as me, which is why I decided to do it by myself. Since I am doing school full-time, I believe the work will be manageable.

My strengths with regards to this are that I have full stack development experience, mainly through my personal portfolio website and a self-hosted database I created. For database design and data preprocessing I will rely mainly on what I have learned through this class, as my other coursework has not covered those topics as much. For deployment and testing and project coordination, I will have to cover these also. As I have a pretty open schedule, I will be able to work on this a lot within my free time along my other projects.

Technology Stack:

SQLite – Very lightweight and will work with Flask well,

HTML, CSS, JavaScript – Basic of a webpage and for making visualizations on the front end through something like charts.js.

Python, Flask – For this I want to have a user and login system and with that I will need some help with flask, and python will help with the calculations and querying the SQLite database. I also need to make sure that my software is secure. This project will be public and so I don't want any security risks.

Github Pages – This would be where the website is hosted

Ngrok – For my setup to work, I will need to create a tunnel so that Github can properly connect to my laptop

Deployment Plan:

My idea for this is to have the front end hosted on GitHub pages. This will allow the user to connect to the website, but that leads to the issue of GitHub's limitations. Specifically, GitHub only allows static pages. To circumvent this, I intend on having GitHub be able to send a request to some deprecated hardware of mine (a really old laptop which I already keep permanently running as a server for other projects) to Flask, and then Flask and the python within will do the query in SQLite on my laptop. For this to work, I will need to use Ngrok to forward a port so that GitHub would be able to connect to the Flask instance. With all of this, anybody who has access to the GitHub, which will be everyone, should have access to the application.

Part 4: References:

Raula, Shandeep. *Retail Supply Chain Sales Dataset*. Kaggle, 2024,

<https://www.kaggle.com/datasets/shandeep777/retail-supply-chain-sales-dataset>.

The ER diagram was created through the use of MySQL Workbench.

Thank you for reading! I'd appreciate any feedback.

Part 5: AI Disclosure Statement

I will only use AI tools for the described use cases, like frontend code, debugging, and UI help. Furthermore, it will assist in general IT work, like port forwarding, ensuring correct connections, the Flask library, etc.

I will not use AI to design the database schema, determine how to normalize, or to generate the SQL queries without proper validation

Signed: Wyatt Keller